

CSEE 4840

Embedded System Design

Lab 3: Peripherals and Device Drivers

Stephen A. Edwards
Columbia University

Spring 2025

Implement on the FPGA a memory-mapped peripheral (an Avalon MM agent) that receives communication from the ARM processors on the Cyclone V. Communicate with your peripheral through a Linux userspace program that accesses a device driver you have written.

Display a ball on the VGA screen with your peripheral. Implement an *ioctl* in your device drive that sends your peripheral coordinates from software.

1 Introduction

In this lab, you will control your own hardware from your own software, communicating through a Linux device driver. We supply a base hardware design to extend, a working example of a VGA peripheral you will have to modify, and a working device driver for it that you will have to adapt to work with your own peripheral.

You will implement a video bouncing ball in this setting. Your peripheral will generate a VGA raster consisting of a ball at a particular location, your userspace C program (software) will make this ball bounce around the screen, and your device driver will mediate between your program and your peripheral.

2 Compile the vga Component Into a New FPGA Image

In this section, you will tell Platform Designer about a new peripheral component, connect it to the ARM processors, and synthesize a new FPGA configuration bitstream.

2.1 Create the vga Ball Component

Download *lab3-hw.tar.gz* from the class website and unpack it on your workstation. Change to the *lab3-hw* directory. Run *qsys-edit soc_system.qsys*, which will bring up a GUI. Its full path is */tools/intel/intelFPGA/21.1/quartus/sopc_builder/bin/*.

Create a new *VGA BALL* component and connect it to the base design. Select File→New Component. This should open the Component Editor window.

In the *Component Type* tab, set Name to *vga_ball* and Display Name to *VGA Ball*.

In the *Files* tab, click *Add File...* under *Synthesis Files* and select the *vga_ball.sv* file. Click on *Analyze Synthesis Files*. This should quickly complete successfully; close the pop-up window. Set *Top-Level Module* to *vga_ball*. Some warnings and errors should appear in the *Messages* tab; we will fix them.

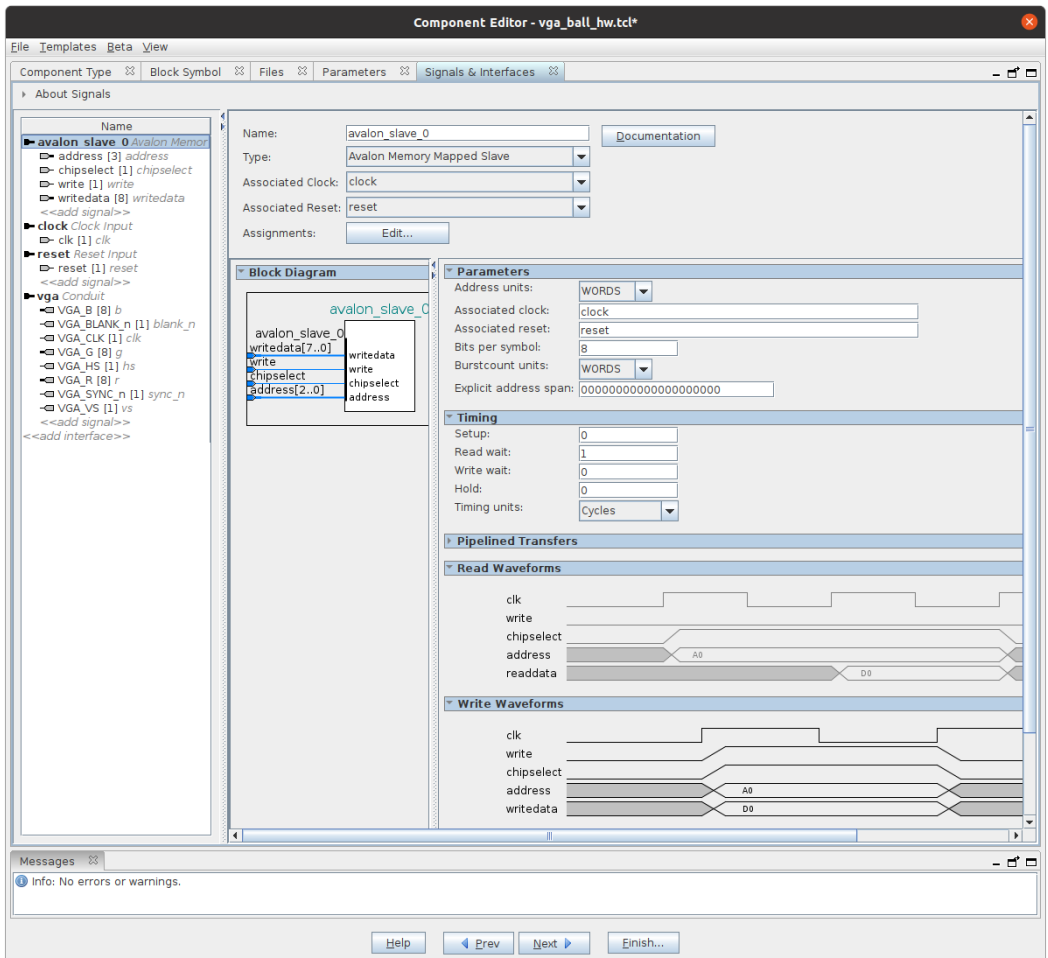
2.2 Assign the Interface Signals on the vga Ball Component

When Platform Designer analyzes the synthesis files, it makes some good guesses about the meaning of each signal on the peripheral, but it is not perfect. Fix the mistakes like this:

Click on the *Signals & Interfaces* tab. Click on *avalon_slave_0* and set its *Associated Reset* to *reset*. Click on *<<add interface>>* in the left box and select *Conduit* (to Platform Designer, a “conduit” is an arbitrary group of signals using an unknown protocol). Set the name of the new conduit to *vga*.

Select and drag all the *vga_* signals so they are under your newly created *vga* conduit. Click on each signal and change its *Signal Type* to a lowercase version of the name after the *vga_*, e.g., *VGA_BLANK_n* should become *blank_n*. Type each of these names.

Your new component should appear in the component editor as shown below. Make sure there are no errors or warnings.



Note that this agent port's address units are set to WORDS, whose size is set by the (must be equal) widths of the *readdata* and *writedata* ports. In the supplied System Verilog, *writedata* is 8 bits wide, so the *address* port delivers byte addresses, just like those used by the processor. If you change *writedata* to 16 bits, words become 16 bits and the *address* port delivers an offset in 16-bit words. The base address continues to point to the first register (whose *address* is 0), but the second register (with *address* 1) will appear in software at the base address plus 2 because functions like *iowrite16()* use byte addresses.

Also notice that the “read wait” timing is set to 1 cycle, meaning that data being read from this component (not used in this lab) is expected the cycle after *chipselect* is asserted, as shown in the timing diagram.

Once you have eliminated all errors, click on *Finish*. It will warn you that it is saving *vga_ball_hw.tcl*; click on *Yes, Save*. The Component Editor window should close.

Open *vga_ball_hw.tcl* with a text editor and add the following three lines after the *module vga_ball* section:

```
set_module_assignment embeddedsw.dts.vendor "csee4840"  
set_module_assignment embeddedsw.dts.name "vga_ball"  
set_module_assignment embeddedsw.dts.group "vga"
```

These make the device show up as compatible with *csee4840,vga_ball-1.0* in the .dtb file, which we will discuss below.

2.3 Connect the VGA Ball Component

Platform Designer now knows about your custom component, so connect it to the rest of your design.

In Platform Designer, add an instance of the new *VGA Ball* component by selecting it under “Project” in the library and clicking on the + Add... button. By default, it will be named *vga_ball_0*.

On the new *vga_ball_0* component instance, connect the clock to *clk* from *clk_0* and connect *reset* to *clk_reset* from *clk_0*.

Connect the *avalon_slave_0* port on *vga_ball_0* to the *h2f_lw_axi_master* port on the *hps_0* component (this is the slower “lightweight” bus from the ARM processors).

Double-click to export *vga_ball_0*’s *vga* conduit in the Export column. Set the name of the export to *vga*. This is the name Platform Designer will use in the generated code.

The System Contents tab should now look like this:

Use	Connections	Name	Description	Export	Clock
☒		clk_0	Clock Source		
☒		clk_in	Clock Input		
☒		clk_in_reset	Reset Input		
☒		clk_reset	Clock Output		
☒		hps_0	Arria V/Cyclone V Hard Proce...		
☒		h2f_user1_clock	Clock Output		
☒		memory	Conduit		
☒		hps_io	Conduit		
☒		h2f_reset	Reset Output		
☒		h2f_axi_clock	Clock Input		
☒		h2f_axi_master	AXI Master		
☒		f2h_axi_clock	Clock Input		
☒		f2h_axi_slave	AXI Slave		
☒		h2f_lw_axi_clock	Clock Input		
☒		h2f_lw_axi_master	AXI Master		
☒		vga_ball_0	VGA Ball		
☒		clock	Clock Input		
☒		reset	Reset Input		
☒		avalon_slave_0	Avalon Memory Mapped Slave		
☒		vga	Conduit		

Current filter:

Messages

Type	Path
2 Info Messages	
1	soc_system.hps_0 HPS Main PLL counter settings: n = 0 m = 73
1	soc_system.hps_0 HPS peripheral PLL counter settings: n = 0 m = 39

0 Errors, 0 Warnings

Generate HDL... Finish

Save the system (File→Save), which should write *soc_system.qsys*.

Generate the Verilog for the design by clicking on *Generate HDL...* (accept the defaults) or running *make qsys*.

Once generating Verilog has completed without warnings or errors, click “Finish” to close Platform Designer.

2.4 Connect the VGA Peripheral to its Pins

Your VGA Ball peripheral needs to communicate through its conduit through pins to an off-chip VGA DAC. To do this, edit *soc_system_top.sv* with a text editor to add the following connections within the instance of *soc_system* near the end of the file:

```
.vga_r (VGA_R),  
.vga_g (VGA_G),  
.vga_b (VGA_B),  
.vga_clk (VGA_CLK),  
.vga_hs (VGA_HS),  
.vga_vs (VGA_VS),  
.vga_blank_n (VGA_BLANK_N),  
.vga_sync_n (VGA_SYNC_N)
```

Platform Designer chose names like *vga_blank_n* by combining the name of the “Export” for the conduit (*vga*, in Platform Designer) with an underscore and the name of each Signal Type when the conduit was defined in the Component Editor.

Delete the two *assign* statements to the VGA signals at the bottom of *soc_system_top.sv*.

2.5 Compile the Hardware Design with Quartus

To compile the Platform Designer-generated Verilog, run *make quartus*. This compiles the System Verilog source (Platform Designer places this in the *soc_system* directory) into an “SRAM object file” *output_files/soc_system.sof*. To do so, it runs the *soc_system.tcl* script to create a preliminary project, then runs the Quartus mapping step to build an initial schematic that enables it to run the *hps_sdram_p0_pin_assignments.tcl* script generated by Platform Designer to configure certain SDRAM-related pins. This process can be done in the Quartus GUI, but it’s far easier to let the provided Makefile do it.

This will generate a lot of warnings. We added a list of innocuous warnings in Platform Designer generated code to a file called *soc_system.srf*. Quartus suppresses these warnings (they’re placed in “Flow Suppressed Messages”). Look carefully at any warnings that are still being generated, especially for those from your files, such as *vga_ball.sv*. All of these are logged in various *.rpt* files found in the *output_files* directory.

2.6 Check the Fmax of the 50 MHz Clock

You only specify cycle-level timing in the synthesizable RTL subset of System Verilog: which signals should change in each cycle, but not their timing during a clock cycle.

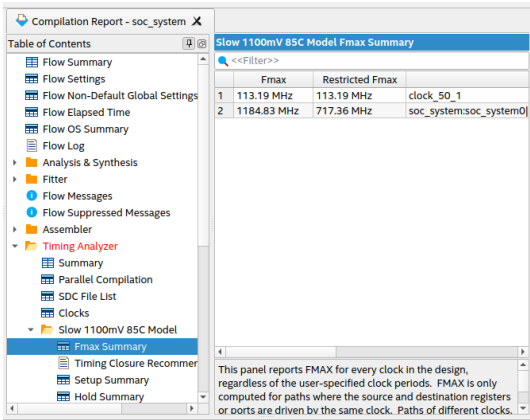
We ask (in `soc_system.sdc`) for a “slow” 50 MHz FPGA clock. Quartus does its best to restructure the logic to meet this, but it may fail on a circuit with too many gates in series.

Quartus performs Static Timing Analysis to verify the generated circuit meets this clock constraint. This involves calculating precise delay numbers for every “gate” and “wire” in the generated circuit and then running an all-paths longest path calculation. Such analysis is standard in modern logic design flows for FPGAs and ASICs.

One key metric is *slack*: the amount of time between when data is guaranteed stable and when the clock may come. If you have negative slack, your data may arrive too late and the circuit is likely to produce incorrect results.

The other key, related metric is *Fmax*, the highest frequency allowed on a given clock before the circuit may start to misbehave.

You can find the results of this analysis in two places. If you run the Quartus GUI, the results of static timing analysis is available in one of the many report windows. For example, *Fmax* is 113 MHz for the skeleton design provided, much higher than the 50 MHz requested.



The screenshot shows the 'Compilation Report - soc_system' window. The 'Table of Contents' on the left lists various reports, with 'Timing Analyzer' expanded to show 'Fmax Summary'. The main panel displays the 'Slow 1100mV 85C Model Fmax Summary' table.

	Fmax	Restricted Fmax	Clock Name
1	113.19 MHz	113.19 MHz	clock_50_1
2	1184.83 MHz	717.36 MHz	soc_system:soc_system0

Below the table, a note states: 'This panel reports FMAX for every clock in the design, regardless of the user-specified clock periods. FMAX is only computed for paths where the source and destination registers or ports are driven by the same clock. Paths of different clocks.'

This number is also reported in `output_files/soc_system.sta.rpt`:

```
+-----+
; Slow 1100mV 85C Model Fmax Summary
+-----+
; Fmax          ; Restricted Fmax ; Clock Name
+-----+
; 113.19 MHz    ; 113.19 MHz      ; clock_50_1
; 1184.83 MHz   ; 717.36 MHz       ; soc_system:soc_system0|soc_system_hps_0:hps_0|soc_
+-----+
```

2.7 Copy *soc_system.rbf* To Your SD Card

After Quartus finishes compiling, convert the *.sof* file to an *.rbf* file by running *make rbf*.

Copy the *output_files/soc_system.rbf* into the boot partition of your SD card. You can mount your SD card on your workstation and copy the file. Alternatively, mount the boot partition by running *mount /dev/mmcblk0p1 /mnt* on your DE1-SoC then use *scp* to copy the file from your workstation to your board, e.g.,

```
scp sedwards@micro11.ee.columbia.edu:lab3/soc_system.rbf /mnt
```

Ensure the file has actually been written out to the card: type *sync* at the command-line.

2.8 Test Your Peripheral from U-Boot (optional)

To isolate hardware from software problems, you can manually exercise your peripheral's registers using *u-boot*, the first stage bootloader. For example, after copying your *.rbf* file to the boot partition, connect your board to your workstation via a mini-USB cable, run `screen /dev/ttyUSB0 115200`, boot your FPGA board, and quickly press a key (such as space) to enter the *u-boot* command line:

```
U-Boot SPL 2013.01.01 (Jan 12 2019 - 19:40:48)
BOARD : Altera SOCFPGA Cyclone V Board
CLOCK: EOSC1 clock 25000 KHz
CLOCK: EOSC2 clock 25000 KHz
...
U-Boot 2013.01.01 (Jan 12 2019 - 19:41:00)

CPU : Altera SOCFPGA Platform
...
Warning: failed to set MAC address

Hit any key to stop autoboot: 0
SOCFPGA_CYCLONE5 #
```

Unless you rebooted from Linux, the FPGA is not yet configured, so read the *.rbf* file, use it to configure the FPGA, and enable the bus bridges:

```
SOCFPGA_CYCLONE5 # fatload mmc 0:1 $fpgadata soc_system.rbf
reading soc_system.rbf
7007204 bytes read in 333 ms (20.1 MiB/s)
SOCFPGA_CYCLONE5 # fpga load 0 $fpgadata $filesize
SOCFPGA_CYCLONE5 # run bridge_enable_handoff
## Starting application at 0x3FF79598 ...
## Application terminated, rc = 0x0
```

Now, you can issue memory write commands to modify registers. Platform Designer put my *vga_ball* agent at address `ff20 0000`, so you can set the red, green, and blue components of the background color using the “memory write” command:

```
SOCFPGA_CYCLONE5 # mw.b ff200000 70
SOCFPGA_CYCLONE5 # mw.b ff200001 d9
SOCFPGA_CYCLONE5 # mw.b ff200002 b3
```

The base addresses of FPGA peripherals begin at `ff20 0000`, which is the base address of the bus bridge. Each peripheral has its own offset beyond that. Base addresses and/or offsets can be found in Platform Designer, in the *soc_system.dts* file, in the */proc/device-tree* directory, or in */proc/iomem* if your kernel driver is installed.

3 Tell the Linux Kernel About Your Peripheral Through the Device Tree

The Linux kernel employs a persistent data structure known as the Device Tree to describe the structure of a hardware platform. It contains information about processors, memory regions, bus bridges, and most importantly, the types and memory location of peripherals such as the VGA Ball. Platform Designer generates a similar *soc_system.sopcinfo* file that, in concert with the *soc_system_board_info.xml* file, can be used to generate an appropriate *soc_system.dtb* file, a binary representation of the Device Tree that is normally loaded as part of the boot process.

Run *embedded_command_shell.sh* to add *sopc2dts* and *dtc* to your path and then generate *soc_system.dtb* by running *make dtb*. These programs are part of the Intel SoC FPGA Embedded Development Suite, which is a separate download from the Intel Quartus website.

Verify that the VGA Ball peripheral appears in the *soc_system.dts* file, which should include

```
vga_ball_0: vga@0x100000000 {  
    compatible = "csee4840,vga_ball-1.0";  
    reg = <0x00000001 0x00000000 0x00000008>;  
    clocks = <&clk_0>;  
}; //end vga@0x100000000 (vga_ball_0)
```

The entry itself comes from the *vga_ball_0* module instance in Platform Designer (*soc_system.qsys*). The *compatible* string is controlled by the *set_module_assignment* statements you should have added to the *vga_ball_hw.tcl* file.

As you did for the *.rbf* file, copy the *soc_system.dtb* file to your SD card's boot partition.

4 Communicate with Your Peripheral Through Software

Connect the console port on your DE1-SoC board (via the mini-USB cable) to your workstation and run `screen /dev/ttyUSB0 115200` as you did in lab 2.

Connect a VGA monitor to your DE1-SoC. Boot Linux on your board from the SD card with your new `soc_system.rbf` and `soc_system.dtb` files (your SD card from lab2 is otherwise fine). If your board is already powered on, restart it by typing `reboot` (don't power-cycle it).

Boot Linux on your board. It should go through the normal boot process and you should see a white box against a colored background on the VGA monitor.

Verify that the kernel sees the VGA Ball device in the device tree:

```
# ls "/proc/device-tree/sopc@0/bridge@0xc0000000/"
#address-cells clock-names compatible ranges reg-names
#size-cells clocks name reg vga@0x100000000
# cat "/proc/device-tree/sopc@0/bridge@0xc0000000/vga@0x100000000/compatible"
csee4840,vga_ball-1.0
```

4.1 Compile and Run the Sample Program

On your board, download and install `linux-headers-4.19.0.tar.gz`, which includes the *Makefile* for compiling kernel modules.

```
# wget https://www.cs.columbia.edu/~sedwards/classes/2025/4840-spring/linux-headers-4.19.0.tar.gz
# tar Pzxf linux-headers-4.19.0.tar.gz
# ls /usr/src/linux-headers-4.19.0
Documentation  arch      drivers  init      mm          scripts    usr
Kconfig        block    firmware ipc        modules.order security  virt
Makefile       certs    fs        kernel    net         sound
Module.symvers crypto   include  lib        samples     tools
```

Install the kernel module mangement programs (e.g., *insmod*, *rmmod*).

```
# apt install -y kmod
```

Download `lab3-sw.tar.gz` from the class website to your board, unpack it, compile it, install the kernel module.

```
# wget https://www.cs.columbia.edu/~sedwards/classes/2025/4840-spring/lab3-sw.tar.gz
# tar xzf lab3-sw.tar.gz
# cd lab3-sw
```

Compile the device driver and user program, install the kernel module, and verify that it works. This should look like

```
# make
make -C /usr/src/linux-headers-4.19.0 SUBDIRS=/root/lab3 modules
make[1]: Entering directory '/usr/src/linux-headers-4.19.0'
CC [M] /root/lab3/vga_ball.o
Building modules, stage 2.
MODPOST 1 modules
CC /root/lab3/vga_ball.mod.o
LD [M] /root/lab3/vga_ball.ko
make[1]: Leaving directory '/usr/src/linux-headers-4.19.0'
cc hello.c -o hello
# insmod vga_ball.ko
# lsmod
Module Size Used by
vga_ball 16384 0
# ./hello
VGA ball Userspace program started
initial state: f9 e4 b7
ff 00 00
00 ff 00
00 00 ff
ff ff 00
...
ff 00 ff
VGA BALL Userspace program terminating
# rmmod vga_ball
rmmod: ERROR: ../libkmod/libkmod.c:514 lookup_builtin_file() could not open
builtin file '/lib/modules/4.19.0/modules.builtin.bin'
```

You may ignore the error from *rmmod*.

make compiles the kernel module (*vga_ball.ko*) and the userspace program (*hello*).

insmod loads the generated kernel module. In the supplied device driver, doing this should change the display. *lsmod* lists installed modules.

The *hello* program is a userspace program that communicates with the *vga_ball* device driver through the *ioctl* system call. It opens the device and reads and writes its state, which changes the color of the background.

rmmod removes the kernel module, which is necessary any time you modify and re-compile the module.

5 What to Do

Modify the hardware and software in the skeleton you have been provided to display a bouncing ball. Change both the interface and contents of the hardware peripheral so that it displays a stationary ball at a software-controllable set of coordinates. Have your peripheral respond to writes to one or more addresses that control the location of the ball.

The register map for the provided vga ball component consists of three single-byte registers, one for each color:

Offset 7 ... 0 Meaning

0	Red	Red component of background color (0–255)
1	Green	Green component of background color (0–255)
2	Blue	Blue component of background color (0–255)

Change this register map so that you can convey (x, y) coordinates of your ball to the hardware. You may modify the width of the agent interface (this is the *writedata* port in *vga_ball.sv*; it is currently 8 bits, but you may want to use 16 or 32) and the number of registers.

Update the comment in *vga_ball.sv* to reflect your new register map.

Record the most conservative F_{max} of your new peripheral (Slow 1100 mV, 85 C) and make sure it is above the required 50 MHz.

Adapt the provided device driver to communicate with your peripheral. E.g., create an *ioctl* that sets the coordinates of the ball.

Write a userspace program that bounces the ball by repeatedly communicating the new coordinates to your peripheral through your device driver.

You may observe that your ball “tears” as it moves across the screen. This is caused by changing the ball’s coordinates while one of its lines is being generated. To fix this, make it so that your ball’s coordinates only change when other lines are being displayed.

6 What to turn in

Find an overworked TA and show him your bouncing ball, your updated register map information in a comment in *vga_ball.sv*, and the F_{max} of your completed project. Once he is satisfied, collect just the files *you wrote or modified* for this lab in a directory called “lab3,” make a tarball with *tar zcf lab3.tar.gz lab3*, and submit that via Courseworks. This should include the SystemVerilog for your peripheral and C source for your device driver and userspace program.

Do not submit everything in your lab3-hw directory: it is too big.

7 Platform Designer Hints

7.1 Editing the Source of Your Platform Designer Component

If you modify the SystemVerilog for your hardware component **without changing its interface**, regenerate your system with Platform Designer then re-run Quartus. Do this by running `make qsys-clean ; make qsys` or open Platform Designer from Quartus (Tools→Qsys) and click on *Generate HDL...*

If you **modify the interface** your hardware component (e.g., to change the number of visible registers, add a read function, or change the signals passed through the conduit), edit the component. Start Platform Designer (e.g., run `qsys-edit`), open your `.qsys` file, select your component under “Project,” and click “Edit.” This should bring up the Component Editor window.

Re-analyze the synthesis files as you did in Section 2.1, make sure the interface signals are assigned correctly, and click *Finish*.

Every time you update the component, re-insert the `set_module_assignment` directives mentioned in Section 2.2.

In Platform Designer, select File→Refresh System (or just press F5). It should complete with a reassuring warning indicating the version of your component has changed. Hovering over the instance of your component should also indicate its version has changed. Save your project after doing this to update the `.qsys` file.

Now, select Generate→Generate... to instruct Platform Designer to regenerate your system so Quartus can recompile it. Alternately, run `make qsys-clean ; make qsys`, which does the same thing from the command line.

7.2 Don't Edit Copies

Do not edit the files in the *synthesis* directory (e.g., in `lab3-hw/synthesis/submodules`). These are copied by Platform Designer and will be overwritten the next time Platform Designer runs.

7.3 Viewing Components as Blocks

Select a component and then View→Block Symbol. This shows how Platform Designer interprets the interface to a component.

